Name: Wajahat ALi Khan

Sap I'd: 55431

Section: BSCS 6-1

Implementing DFS for the following graph:

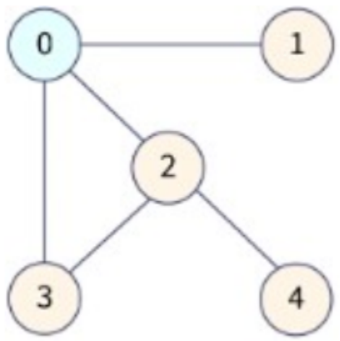
0: [1,2,3]

1: [0]

2: [0,3,4]

3: [0,3]

4: [2]




```
def dfs(node, graph, visited, component):
    component.append(node)
    visited[node] = True

    for child in graph[node]:
        if not visited[child]:
            dfs(child, graph, visited, component)


if __name__ == "__main__":

    graph = {
        0: [1, 2, 3],
        1: [0],
        2: [0, 3, 4],
        3: [0, 3],
        4: [2]
    }
    node = 0
    visited = [False] * len(graph)
    component = []
    dfs(node, graph, visited, component)
```

```
        dfs(node, graph, visited, component)
        print(f"Following is the Depth-first search: {component}")
```

```
Following is the Depth-first search: [0, 1, 2, 3, 4]
```

Implementing DFS for the following graph:

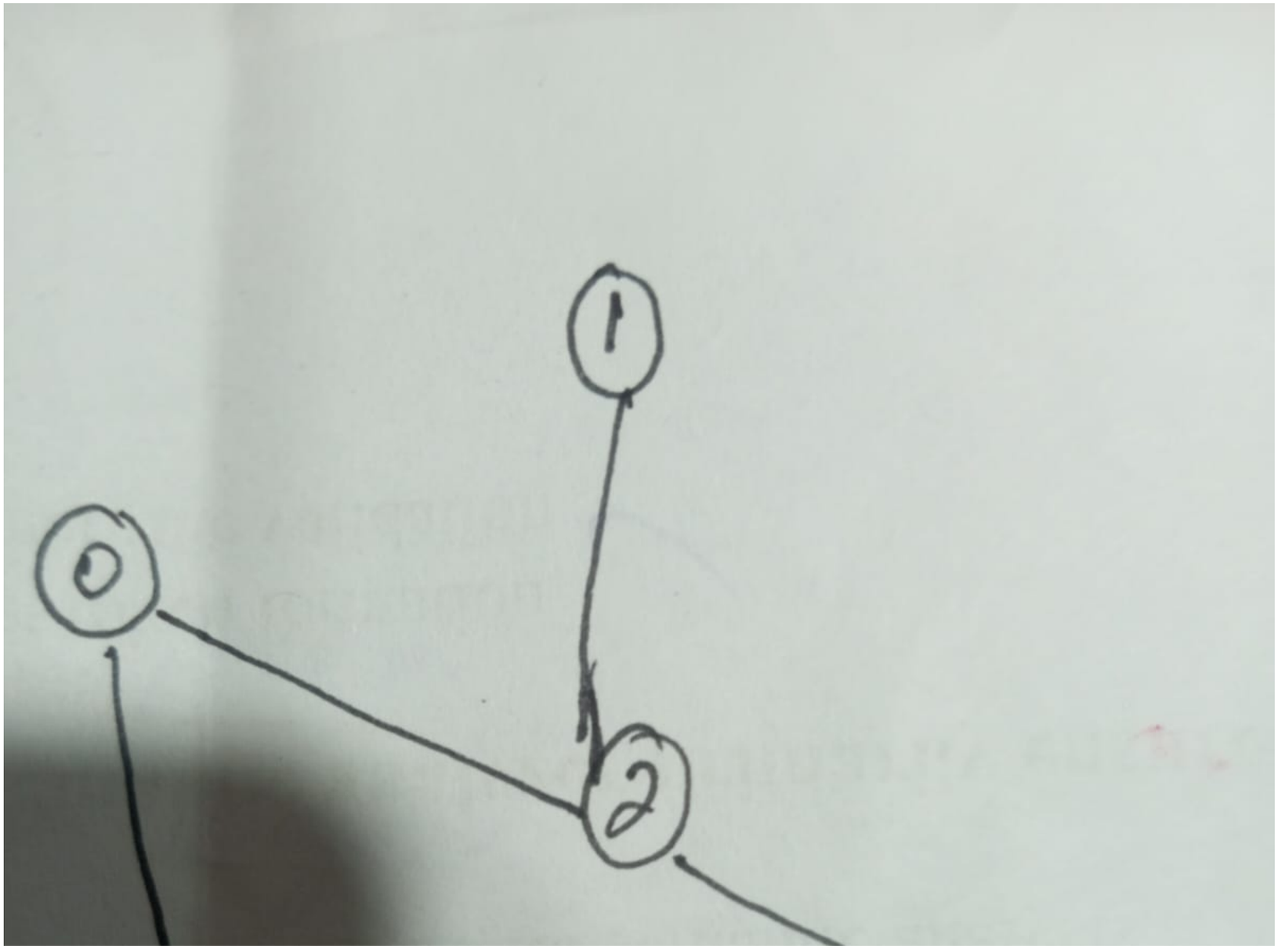
0: [2,3]

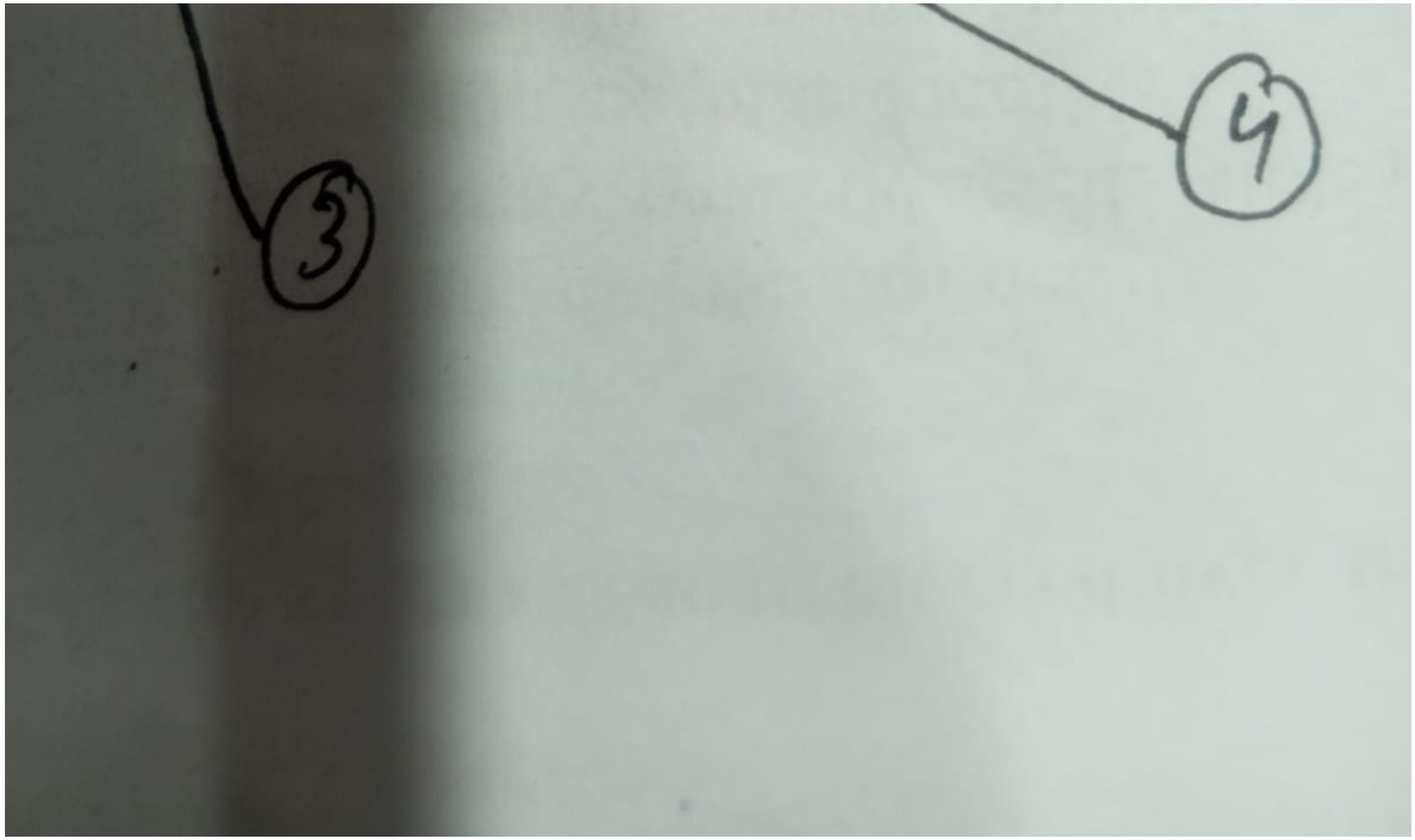
1: [2]

2: [0, 4]

3: [0]

4: [2]






```
def dfs(node, graph, visited, component):
    component.append(node)
    visited[node] = True

    for child in graph[node]:
        if not visited[child]:
            dfs(child, graph, visited, component)


if __name__ == "__main__":

    graph = {
        0: [2, 3],
        1: [2],
        2: [0, 1, 4],
        3: [0],
        4: [2]
    }
    node = 0
    visited = [False] * len(graph)
    component = []
```

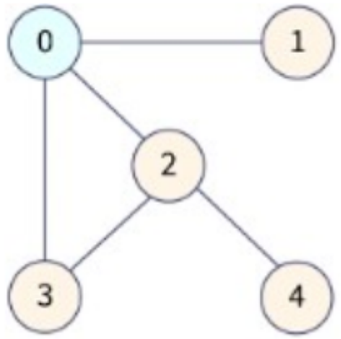
```
        dfs(node, graph, visited, component)
        print(f"Following is the Depth-first search: {component}")
```

```
Following is the Depth-first search: [0, 2, 1, 4, 3]
```

Implementing BFS for the following graph:

0: [1,2,3]

1: [0]



2: [0,3,4]

3: [0,3]

```
from collections import deque

def bfs(start_node, graph, visited, component):
    queue = deque([start_node])  # Initialize queue with starting node
    visited[start_node] = True  # Mark starting node as visited

    while queue:  # Continue until queue is empty
        node = queue.popleft()  # Remove and return the leftmost element
        component.append(node)  # Add node to traversal order

        # Visit all neighbors of the current node
        for child in graph[node]:
            if not visited[child]:  # If neighbor not visited
                visited[child] = True  # Mark as visited
                queue.append(child)  # Add to queue for later processing


if __name__ == "__main__":
    # Graph of nodes
    graph = {
        0: [1, 2, 3],
        1: [0],
        2: [0, 3, 4],
        3: [0, 2],  # Fixed: changed [0, 3] to [0, 2] (removed self-loop)
        4: [2]
    }
    node = 0  # Starting node
    visited = [False] * len(graph)  # Make all nodes to False initially
```

```
        component = []
        bfs(node, graph, visited, component)  # Traverse using BFS
        print(f"Following is the Breadth-first search: {component}")
```

```
Following is the Breadth-first search: [0, 1, 2, 3, 4]
```